

Indian Statistical Institute
BSDS, First Year, First Semester Examination, 2025-26
Introduction to Computing

Full Marks: 50

Date: 15-12-2025

Time: 3 Hours

Answer any *five* of the following questions

$5 \times 10 = 50$

1. Write down the output of each of the following Python code segment. Justify your answer.

- (a)

```
i = 10
while i%1 == 0:
    if i%3 == 0:
        break
    print (i)
    i -= 1
```
- (b)

```
def find(ls):
    if len(ls) == 1:
        print (ls)
        return
    ls.remove(min(ls))
    find(ls)
find([7, 4, 10, 2, 9])
```
- (c)

```
s1, s2 = "often", "tensed"
target_i = 0
for i in range(1, min(len(s1), len(s2)) + 1):
    if s1[-i:] == s2[:i]:
        target_i = i
print (s2[:target_i])
```
- (d)

```
str, output = "tenet", []
done = set()
for ch in str:
    if ch not in done:
        done.add(ch)
        output.append(ch)
print (output)
```
- (e)

```
result = lambda f1: lambda f2: f1 / f2
result = result(1)
print (result(0.1))
```

[2+2+2+2+2]

2. The following Python program finds out all the prime numbers from 2 up to n. Write an efficient version of this code. Highlight the portions that are efficient and justify your answer. You will receive higher credit if your program is more efficient.

```

import math
n = int(input())
primes = []
for num in range(2, n+1):
    is_prime = True
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        primes.append(num)
print ("Primes up to", n, ":", primes)

```

[10]

3. Write an object-oriented version of the following Python program that aims to implement a *Queue* data structure. Note that a queue data structure follows the FIFO principle. You will receive higher credit if you use more object-oriented features in your program.

```

def start_queue(size=10):
    queue = []
    max_size = size
    print ("Queue initialized")
    return queue, max_size
def queue_insert(queue, max_size, data):
    if len(queue) < max_size:
        queue.insert(0, data)
    else:
        print ("Queue overflow!!!")
def queue_delete(queue):
    if len(queue) > 0:
        data = queue.pop()
        return data
    else:
        print ("Queue underflow!!!")
        return None
def queue_display(queue):
    print ("The elements in the queue are:")
    for element in queue:
        print(element)
def finish_queue(queue):
    queue.clear()
    print ("Queue destructed")
queue, max_size = start_queue()
finish_queue(queue)

```

[10]

4. (a) Recall that *binary search* takes a sorted array, say `array`, as input and finds the index of a target element, say `target`, therein by repeatedly halving the search range. If `target` is not found in `array`, it returns the index `-1`. The following erroneous Python function aims to implement binary search. Correct it and justify your answer.

```
def Bsearch(array, target):
    low = 0
    high = len(array) - 1
    while low < high:
        mid = (low + high)/2
        if array[mid] == target:
            return mid
        elif array[mid] < target:
            low = mid + 1
        else:
            mid = high + 1
    return -1
```

In what format the array argument is to be passed to the Bsearch() function?

(b) What is the purpose of the following Python code? Justify your answer.

```
def function(ls):
    if not ls:
        return [[]]
    result = function(ls[1:])
    return result + [[ls[0]] + s for s in result]
str = input()
ls = function(list(str))
n = len(ls)
for i in range(n):
    for j in range(0, n-i-1):
        if len(ls[j]) > len(ls[j + 1]):
            ls[j], ls[j + 1] = ls[j + 1], ls[j]
print (ls)
```

[(3+1)+6]

5. (a) Evaluate the following postfix expression:

$7\ 3 - 1\ 2 + 3 / *$

Show the conversion step by step by using an appropriate data structure.

(b) Suppose the in-order and post-order traversal results obtained from a binary tree are given below.

In-order: 7, 3, 2, 5, 11

Post-order: 2, 3, 7, 5, 11

Based on the above, build the original binary tree and provide explanation for each step of construction. Is an alternative construction of the tree possible besides the one you have built?

[4+(5+1)]

6. Mention and justify whether the following statements are TRUE or FALSE.

(i) $f(n) = 12n^2 + 5n\log n + 1729$ is $O(n^2)$.

(ii) $f(n) = 5n\log n + (\log n)^2 + \log n^2$ is $\Omega(n\log n)$.

(iii) If $f_1(n)$ is $\Theta(g_1(n))$ and $f_2(n)$ is $\Theta(g_2(n))$, then $f_1(n) * f_2(n)$ is $\Theta(g_1(n) * g_2(n))$.

(iv) If $f_1(n)$ is $O(g_1(n))$ and $f_2(n)$ is $O(g_2(n))$, then $f_1(n) + f_2(n)$ is $O(\min(g_1(n), g_2(n)))$.

(v) If $f(n)$ is $\Omega(g(n))$ and $g(n)$ is $\Omega(h(n))$, then $f(n)$ is $\Omega(h(n))$.

[2+2+2+2+2]

7. (a) What is the runtime complexity of quicksort in the following cases? Justify your answer.
- (i) All the elements in the input array are same.
 - (ii) All the elements in the input array are distinct and sorted in decreasing order.
- (b) What is the most practically suitable sorting algorithm in your opinion? Justify your answer.

[(4+4)+2]
